SIMATS ENGINEERING

Department of Information Technology

CSA-1410 COMPILER DESIGN – UNIT III: SYNTAX-DIRECTED TRANSLATION

ASSIGNMENT 3 – CO3

Syntax-Directed Translation: Synthesized and Inherited Attributes, Dependency Graphs, SDT Schemes and Attribute Evaluation

Details	Information
Student Name	Ch.V.Ramayogi
Student B Register No.	192424319
Department / Section	B.E-AIDS/SLOT-D
Academic Year / Semester	Sep–Dec 2026
Faculty Name	Dr. N. Deepa

Submitted in partial fulfilment of the requirements for Compiler Design – Unit III.

CONTENTS

- Q1 Synthesized Attributes and Annotated Parse Tree
- Q2 Inherited Attributes in Declarations
- Q3 Dependency Graphs and Evaluation Order
- Q4 Syntax-Directed Translation Schemes
- Q5 Parse Tree and Abstract Syntax Tree Construction
- Q6 Advanced Attribute Evaluation
- Q7 Design Challenge
- Contribution Record
- Conclusion
- References

Notation: E.val denotes the synthesized numeric value of expression E. L.in and T.type denote inherited and synthesized attributes respectively. ε denotes the empty string. All arithmetic examples use integer arithmetic; division is assumed to be exact for the selected examples.

Q1 – SYNTHESIZED ATTRIBUTES AND THE ANNOTATED PARSE TREE (6 MARKS)

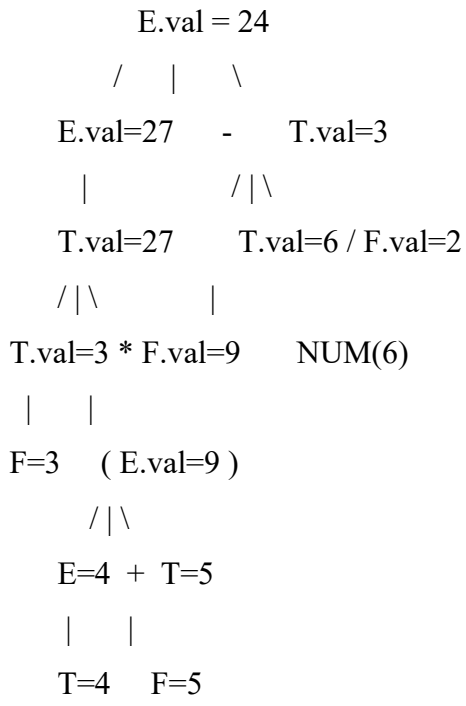
1a. Shared: SDD for arithmetic expressions

A syntax-directed definition (SDD) associates semantic rules with grammar productions. In this question, each expression node synthesizes its value from the values of its children. The grammar gives multiplication and division higher precedence than addition and subtraction, and parentheses override precedence.

Productions	Semantic rules
E → E1 + T	E.val = E1.val + T.val
E → E1 - T	E.val = E1.val - T.val
E → T	E.val = T.val
T → T1 * F	T.val = T1.val * F.val
T → T1 / F	T.val = T1.val / F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → NUM	F.val = NUM.lexval

NUM.lexval is supplied by the lexical analyser. E.val, T.val and F.val are synthesized because information flows from children to parent.

Parse of 3 * (4 + 5) - 6 / 2



Evaluation: 4 + 5 = 9; 3 × 9 = 27; 6 ÷ 2 = 3; 27 − 3 = 24. Therefore the root attribute is E.val = 24.

Bottom-up parser/evaluator pseudocode

```
parseE():
    left = parseT()
    while lookahead is '+' or '-':
        op = consume()
        right = parseT()
        if op == '+': left = Node('+', left, right, left.val + right.val)
        else:         left = Node('-', left, right, left.val - right.val)
    return left
parseT():
    left = parseF()
    while lookahead is '*' or '/':
        op = consume()
        right = parseF()
        if op == '*': left = Node('*', left, right, left.val * right.val)
        else:
            if right.val == 0: report division-by-zero
            left = Node('/', left, right, left.val / right.val)
    return left
parseF():
    if lookahead is NUM:
        n = consume(NUM); return Leaf(n.lexval)
    if lookahead == '(':
        consume('('); e = parseE(); consume(')'); return e
    report syntax error
```

The parser is bottom-up in its attribute computation: a parent value is assigned only after its child values are available. The pseudocode uses loops to implement left associativity while preserving the grammar's precedence.

1b. Student A – annotated tree and S-attributed proof

Every internal node is annotated with the value obtained from its children. The parse tree above gives the following post-order computation sequence:

1. NUM(3) gives F.val = 3, then T.val = 3.
2. NUM(4) gives F.val = 4, then T.val = 4.

3. NUM(5) gives F.val = 5; E.val = 4 + 5 = 9; parent F.val = 9.
4. T.val = 3 × 9 = 27; then E.val = 27.
5. NUM(6) gives F.val = 6 and T.val = 6.
6. NUM(2) gives F.val = 2; T.val = 6 / 2 = 3.
7. Root E.val = 27 − 3 = 24.

An attribute is synthesized when it is computed at a node using attributes of that node's children (and lexical values at leaves). In this SDD, every semantic rule computes E.val, T.val or F.val from child attributes only. No parent or sibling sends information downward. Hence the SDD is S-attributed.

Post-order is forced by the dependencies: a parent cannot be evaluated until the values of its child subtrees are known. Any valid dependency-respecting order may evaluate independent child subtrees in either order, but each parent must follow its children.

1c. Student B – test expressions and instrumentation

No.	Expression	Expected result	Status
1	3+4	7	Valid
2	8-3	5	Valid
3	2*6	12	Valid
4	9/3	3	Valid
5	2+3*4	14	Valid
6	(2+3)*4	20	Valid
7	20-6/2	17	Valid
8	3*(4+5)-6/2	24	Valid
9	7+8*2-3	20	Valid
10	18/(3*2)	3	Valid
11	4+*2	—	Syntax error
12	5/(3-3)	—	Division by zero

Instrumentation records one event whenever a syntax-tree node is created and one attribute computation whenever a semantic value is assigned. For a binary operator node there is one synthesized value; for a numeric leaf there is one lexical value. A hand calculation is compared with the program result for every valid test. The two invalid tests must be rejected rather than assigned a misleading numeric value.

Expression	Hand result	Program result	Match
2+3*4	14	14 (expected)	Yes
(2+3)*4	20	20 (expected)	Yes
3*(4+5)-6/2	24	24 (expected)	Yes
18/(3*2)	3	3 (expected)	Yes
4+*2	Syntax error	Syntax error	Yes
5/(3-3)	Division by zero	Division by zero	Yes

Count relationship: for a binary expression tree with n numeric leaves and only binary operators, the tree contains n−1 operator nodes and 2n−1 total expression nodes. Each expression node has one synthesized value, so the number of value computations is proportional to the number of nodes: O(n). Parentheses may add parse-tree punctuation nodes, but they need not create separate AST nodes.

1d. Height attribute and recursion comparison

E → E1 + T E.height = 1 + max(E1.height, T.height)

E → E1 - T E.height = 1 + max(E1.height, T.height)

E → T E.height = T.height

T → T1 * F T.height = 1 + max(T1.height, F.height)

T → T1 / F T.height = 1 + max(T1.height, F.height)

T → F T.height = F.height

F → (E) F.height = E.height

F → NUM F.height = 1

Height is synthesized just like value. A leaf has height 1. Each operator node has one plus the maximum height of its children. A left-recursive grammar naturally creates a left-leaning parse-tree spine for long chains such as a−b−c−d. A right-recursive grammar creates a right-leaning spine. Both represent the same associativity only if the semantic actions preserve the intended grouping.

Number of operands	Left-associative chain height (operator levels)	Right-recursive chain height (operator levels)
2	1	1
3	2	2
4	3	3
5	4	4

6	5	5
8	7	7

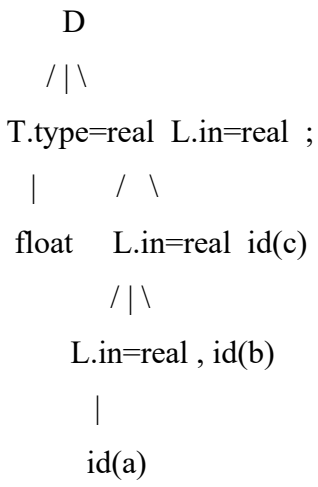
For a simple unbalanced chain, height grows linearly with expression length in either recursive direction; the direction changes which side contains the long spine, not the asymptotic height. A balanced expression tree can have logarithmic height. The following experiment should record measured heights from generated trees rather than treating the table as measured runtime data.

Q2 – INHERITED ATTRIBUTES IN DECLARATIONS (7 MARKS)

2a. Shared SDD for declarations

Productions	Semantic rules
D → T L ;	L.in = T.type
T → int	T.type = integer
T → float	T.type = real
L → L1 , id	L1.in = L.in; addType(id.entry, L.in)
L → id	addType(id.entry, L.in)

For the declaration float a, b, c, the type node synthesizes T.type = real. This value is passed as the inherited attribute L.in to the identifier list. Each identifier receives the same type.



Symbol-table insertions:

addType(a, real)

addType(b, real)

addType(c, real)

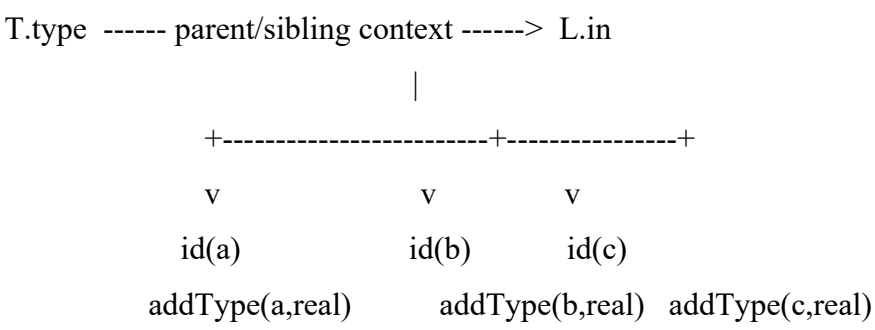
addType routine

```
addType(name, type):
   if name already exists in the current scope:
      report "duplicate declaration: " + name
      return
   insert { name, type, current_scope } into symbol_table
```

2b. Student A – information flow and evaluation

L.in cannot be synthesized in this grammar because the type is determined by the sibling T, not by the identifier-list subtree. The list itself contains names but does not independently know whether the declaration began with int or float. The parent declaration passes the type information down to L.

Information flow:



The required evaluation order is: first evaluate T.type; then assign L.in; then visit each identifier and call addType. The type must be known before inserting any identifier.

Left- and right-recursive list forms

Left-recursive:

L → L1 , id L1.in = L.in; addType(id.entry, L.in)

L → id addType(id.entry, L.in)

Right-recursion-eliminated:

L → id L'

L' → , id L'1

L' → ε

For the right-recursive form, pass the same inherited type:

L'.in = L.in

At L' → , id L'1:

addType(id.entry, L'.in)

L'1.in = L'.in

At L' → ε:

no insertion

In both forms, the declaration's type is passed along the list. The left-recursive form propagates the inherited value to the left list child; the right-recursive form propagates it to the continuation L'.

Symbol table example

Identifier	Type	Scope	Result
a	real	global	Inserted
b	real	global	Inserted
c	real	global	Inserted

2c. Student B – synthesized versus inherited

Attribute kind	Direction	Typical use here	Main benefit / limitation
Synthesized	Child → parent	T.type is computed from the type keyword.	Natural for bottom-up evaluation; cannot directly carry a parent's context down.
Inherited	Parent or left sibling → child	L.in receives the type selected by T.	Carries context such as declared type; evaluation depends on context being available.

The same type information could be represented differently by changing the grammar or using a global declaration context, but the inherited attribute expresses the relationship explicitly and safely. If L.in is read before it is assigned, the scanner/parser may insert identifiers with an undefined type or produce incorrect semantic information.

Duplicate declaration detection occurs in addType by checking the current scope before insertion. Undeclared identifier detection occurs later when an identifier is used: lookup(name) must succeed in the current or enclosing scope; otherwise, report an undeclared-variable error.

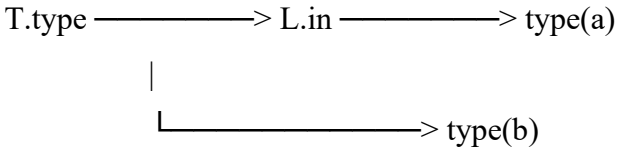
Joint per-variable type design: each symbol-table entry stores its own type. For example, int a; float b; produces a→integer and b→real. The inherited type is local to each declaration and is not permanently shared between declarations.

Q3 – DEPENDENCY GRAPHS AND EVALUATION ORDER (7 MARKS)

3a. Shared dependency graphs

A dependency graph contains one vertex for each attribute instance. An edge X → Y means that Y cannot be evaluated until X has been evaluated. A valid evaluation order is any topological ordering of this directed graph.

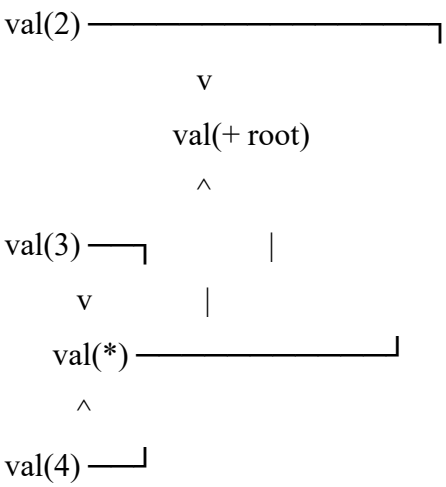
Declaration: float a, b



One valid order:

T.type, L.in, type(a), type(b)

Expression: 2 + 3 * 4



More explicitly:

val(3), val(4) -> val(*)

val(2), val(*) -> val(+)

Valid order: val(2), val(3), val(4), val(*), val(+)

The root value is 2 + (3 × 4) = 14.

Generic attribute evaluator

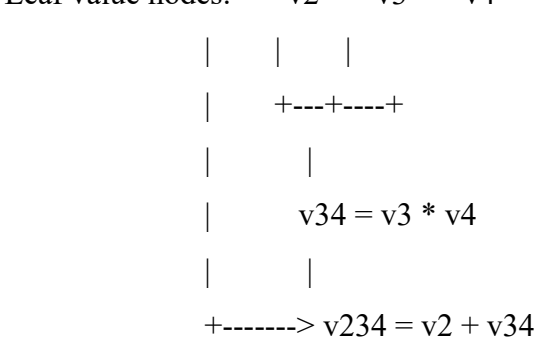
```
evaluate(attributes, dependency_edges):
    indegree[v] = number of incoming edges for every attribute v
    ready = all vertices whose indegree is zero
    order = []

    while ready is not empty:
        remove one vertex v from ready
        evaluate the semantic rule for v
        append v to order
        for each edge v -> w:
            indegree[w] = indegree[w] - 1
            if indegree[w] == 0:
                add w to ready

    if length(order) != number_of_vertices:
        report "cyclic attribute dependencies"
    else:
        return order
```

3b. Student A – graph on tree, alternative orders and cycle detection

For 2 + 3 * 4, the parse structure is E(2, +(root), T(3 * 4)). The dependencies are:



Topological order 1: v2, v3, v4, v34, v234

Topological order 2: v4, v2, v3, v34, v234

Both orders are valid because the leaf attributes v2, v3 and v4 are independent. The multiplication node must follow v3 and v4, and the root addition must follow v2 and v34. Every valid order gives 14 because the dependency constraints, not the arbitrary order among independent nodes, determine correctness.

Cyclic SDD example

$A \rightarrow B \quad A.x = B.x + 1$

$B \rightarrow A \quad B.x = A.x + 1$

The two attributes depend on each other: A.x → B.x and B.x → A.x. There is no vertex with indegree zero, so Kahn's topological-sort algorithm cannot remove any node. If the number of processed vertices is smaller than the total number of vertices, the dependency graph contains a cycle and the attributes cannot be evaluated using a simple one-pass dependency order.

3c. Student B – S-attributed and L-attributed definitions

Class	Formal condition	Evaluation strategy
S-attributed	All attributes are synthesized.	Evaluate bottom-up, typically during reductions or post-order traversal.
L-attributed	For production A → X1 X2 ... Xn, each inherited attribute of Xi may depend only on inherited attributes of A and attributes of X1...X(i−1).	Evaluate left-to-right in a depth-first traversal; parent context and left siblings are available.

Q1 is S-attributed because every expression value is synthesized from children. Every S-attributed definition is also L-attributed. Q2 is L-attributed: L.in is passed from the declaration context (and propagated through the list), while T.type is synthesized before it is passed. Q2 is not S-attributed because it contains an inherited attribute.

Example of L-attributed but not S-attributed: D → T L with L.in = T.type. The inherited attribute is allowed because T is a left sibling of L.

Example that is not L-attributed: A → B C where B.in depends on C.synth. C is a right sibling, so the inherited attribute of B depends on a symbol to its right, violating the L-attributed condition.

Evaluation strategy: S-attributed definitions use post-order/bottom-up evaluation; L-attributed definitions use left-to-right depth-first evaluation; a general non-L-attributed definition may require a dependency

graph and a topological order, or a redesign of the grammar. Restricting a practical compiler to L-attributed definitions simplifies one-pass left-to-right parsing and makes inherited information available when needed.

Q4 – SYNTAX-DIRECTED TRANSLATION SCHEMES (7 MARKS)

4a. Shared: SDD versus SDT and postfix translation

An SDD is a grammar together with attributes and semantic rules. An SDT places executable semantic actions directly inside the right-hand side of grammar productions. An SDD describes what values are computed; an SDT specifies when the actions execute during parsing.

Grammar and postfix SDD:

E → E1 + T E.post = E1.post || T.post || "+"

E → E1 - T E.post = E1.post || T.post || "-"

E → T E.post = T.post

T → T1 * F T.post = T1.post || F.post || "*"

T → T1 / F T.post = T1.post || F.post || "/"

T → F T.post = F.post

F → (E) F.post = E.post

F → id F.post = id.lexeme

Postfix SDT

E → E + T { print("+"); }

E → E - T { print("-"); }

E → T

T → T * F { print("*"); }

T → T / F { print("/"); }

T → F

F → (E)

F → id { print(id.lexeme); }

The operand action is placed when the identifier is recognised. The operator action is placed after its right operand, so the operands are emitted before the operator.

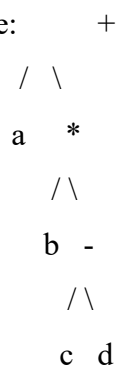
For a + b * (c - d), the postfix result is: a b c d - * +.

Infix expression	Postfix output
a+b	a b +
a+b*c	a b c * +
(a+b)*c	a b + c *
a-b-c	a b - c -
a/(b+c)	a b c + /
a+b*(c-d)	a b c d - * +

4b. Student B – action placement and execution

In a bottom-up parser, a production's action placed at the end is executed when the parser reduces that production. This is suitable for postfix translation because the left and right operand text has already been processed before the operator is emitted.

Parse structure:



Action sequence:

1. emit a
2. emit b
3. emit c
4. emit d
5. reduce c-d; emit -
6. reduce b*(c-d); emit *
7. reduce a + [result]; emit +

A mid-production action executes before the parser has completed the entire production. For example, E → E1 + { action } T executes after the plus sign has been recognised but before T is parsed. This is useful for inherited information or opening a scope, but it cannot emit the plus operator in postfix order because the right operand has not yet been emitted.

4c. Student A – tests, action counts and left recursion

Test purpose	Expression	Expected postfix
Precedence	a+b*c	a b c * +
Parentheses	(a+b)*c	a b + c *
Left associativity	a-b-c	a b - c -
Right grouping via brackets	a-(b-c)	a b c - -
Division precedence	a/b+c	a b / c +
Nested brackets	a*(b+(c-d))	a b c d - + *

For an expression with n operands and n−1 binary operators, a postfix translator emits n operand actions and n−1 operator actions, giving 2n−1 output actions. The action count grows linearly with expression size. Compared with Q1, Q1 computes numeric attributes at reductions; this SDT emits output text at the appropriate recognition/reduction point.

Removing left recursion changes the grammar shape. For example, E → E + T | T becomes E → T E' and E' → + T E' | ε. The right-recursive tail must carry the accumulated left expression as an inherited attribute (E'.inh) to preserve left associativity. If the action is placed incorrectly, a−b−c may become a−(b−c), changing the meaning.

On-the-fly translation is appropriate when the output can be emitted as soon as the required operands are known. It is less suitable when later context can change the meaning, when extensive optimisation is required, or when the complete syntax tree must be inspected before output.

Q5 – GENERATING THE PARSE TREE / ABSTRACT SYNTAX TREE USING ATTRIBUTES (6 MARKS)

5a. Shared: AST-building SDD

A parse tree contains all grammar symbols and punctuation. An abstract syntax tree (AST) keeps only the essential language constructs. Each node attribute carries a pointer to a tree node. The function mkNode creates an operator or assignment node and connects its children.

E → E1 + T E.node = mkNode("+", E1.node, T.node)

E → E1 - T E.node = mkNode("-", E1.node, T.node)

E → T E.node = T.node

T → T1 * F T.node = mkNode("*", T1.node, F.node)

T → T1 / F T.node = mkNode("/", T1.node, F.node)

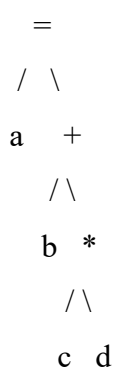
T → F T.node = F.node

F → (E) F.node = E.node

F → id F.node = mkLeaf(id.lexeme)

S → id = E S.node = mkNode("=", mkLeaf(id.lexeme), E.node)

For a=b+c*d, the AST is:



The parse tree also contains E, T, F and punctuation such as parentheses. The AST removes these grammar-only nodes and retains the assignment and arithmetic operators and identifiers.

5b. Student A – pointer passing and inherited E'.inh

A child constructs its own node and returns the pointer in its synthesized node attribute. The parent receives the child pointers and creates a new parent node. No complete subtree needs to be copied; only pointers are connected.

Left-recursive grammar

E → E1 + T E.node = mkNode("+", E1.node, T.node)

E → E1 - T E.node = mkNode("-", E1.node, T.node)

E → T E.node = T.node

T → T1 * F T.node = mkNode("*", T1.node, F.node)

T → T1 / F T.node = mkNode("/", T1.node, F.node)

T → F T.node = F.node

F → (E) F.node = E.node

F → id F.node = mkLeaf(id.lexeme)

Left-recursion-eliminated grammar

E → T E'

E' → + T E'1 | - T E'1 | ε

T → F T'

T' → * F T'1 | / F T'1 | ε

E.inh = T.node

E'.inh carries the accumulated expression node.

For E' → + T E'1:

temp = mkNode("+", E'.inh, T.node)

E'1.inh = temp

For E' → - T E'1:

temp = mkNode("-", E'.inh, T.node)

E'1.inh = temp

For E' → ε:

E'.node = E'.inh

E'.inh is inherited because it carries the already-built left-hand expression from the parent/previous part of the production into the recursive tail. Without it, the tail would know only the next term and could not combine it with the accumulated expression.

Step-by-step AST for a = b + c * d

8. Create leaf nodes a, b, c and d as their identifiers are recognised.
9. Create multiplication node *(c,d), because multiplication has higher precedence.
10. Create addition node +(b, *(c,d)).
11. Create assignment node =(a, +(b, *(c,d))).
12. Return the assignment node pointer as the root of the AST.

5c. Student B – node-count and memory comparison

Construct	Typical nodes for a=b+c*d	Reason
Parse tree	Many more nodes: grammar nonterminals E, T, F plus punctuation and terminals.	Represents every grammar production and token.
AST	7 essential nodes: assignment, a, +, b, *, c, d.	Retains only operators and operands.

For the expression subtree b+c*d, the AST has 5 nodes: +, b, *, c and d. The assignment adds = and a, giving 7 nodes. A parse tree has additional grammar nodes for E, T and F and may include parentheses and separators. AST size grows linearly with the number of operands/operators but has a smaller constant

factor. Fewer nodes usually mean less memory and faster later traversals. A's cross-explanation: inherited E'.inh is needed because the transformed grammar processes a continuation that must carry the accumulated left subtree. B's cross-explanation: AST compactness comes from removing grammar-only nodes, not from removing meaningful operators or operands.

Q6 – ADVANCED ATTRIBUTE EVALUATION MINI-INVESTIGATIONS (7 MARKS)

6a. Student A – inherited attributes in bottom-up LR parsing

In an LR parser, reductions occur after the right-hand side has been recognised. A synthesized attribute is straightforward because it can be computed from values already on the stack. An inherited attribute may depend on a parent or left sibling that is not naturally available at the point when a right-hand-side symbol is first shifted. Therefore inherited attributes are not directly evaluated in the same simple way as synthesized attributes.

Marker-nonterminal technique

Original declaration:

D → T L ;

T → int | float

L → L , id | id

Transformed grammar with marker M:

D → T M L ;

M → ε

T → int | float

L → L , id | id

At M → ε, the parser has already recognised T.

The semantic action stores the current declaration type in context.

When each id is reduced, it receives that stored type and is inserted.

The marker M is an ε-production: it consumes no input and does not add or remove any terminal strings. It only provides a safe point in the parse where the type from T is available before parsing the identifier list.

Illustrative stack/action trace for int x, y

Step	Parser recognition / reduction	Semantic state
1	Shift int	Type token recognised.
2	Reduce T → int	T.type = integer.
3	Reduce M → ε	current_type = integer.
4	Shift x; reduce L → id	addType(x, integer).
5	Shift comma and y; reduce L → L , id	addType(y, integer).
6	Reduce D → T M L ;	Declaration complete.

A parser implementation can store current_type in a semantic-value field or in a marker nonterminal's value. Fixed-position stack access is possible only when the parser generator's stack layout and reduction length are known and stable. It is fragile: grammar changes, optional symbols or different parser-generator conventions can change offsets. A named marker attribute is safer.

The technique is not suitable when the required context is not available before the marker, depends on a right sibling that has not been parsed, or must vary independently for nested declarations. In those cases the grammar or translation design must be revised.

6b. Student B – three-address code SDD

Three-address code (TAC) represents each operation using at most three addresses. E.addr stores the location containing the expression result. E.code stores the sequence of instructions generated for that expression. newTemp() returns a fresh temporary name.

E → E1 + T

E.addr = newTemp()

E.code = E1.code || T.code ||

[E.addr = E1.addr + T.addr]

E → E1 - T

E.addr = newTemp()

E.code = E1.code || T.code ||

[E.addr = E1.addr - T.addr]

T → T1 * F

T.addr = newTemp()

T.code = T1.code || F.code ||

[T.addr = T1.addr * F.addr]

T → T1 / F

T.addr = newTemp()

T.code = T1.code || F.code ||

[T.addr = T1.addr / F.addr]

F → id

F.addr = id.lexeme

F.code = []

F → NUM

F.addr = NUM.lexval

F.code = []

TAC for x = a + b * c - d

t1 = b * c

t2 = a + t1

t3 = t2 - d

x = t3

Temporary numbering: t1 is created for b*c; t2 for a+t1; t3 for t2−d. The multiplication is generated first because it has higher precedence. The final assignment copies the expression result to x.

If-else with inherited labels

S → if (B) S1 else S2

B.true = newLabel()

B.false = newLabel()

S1.next = newLabel()

S2.next = S.next

emit(B.code)

emitLabel(B.true)

emit(S1.code)

```
emit("goto " + S.next)
emitLabel(B.false)
emit(S2.code)
emitLabel(S.next)
```

Labels such as B.true, B.false and S.next are inherited because they are control-flow destinations supplied by the enclosing statement context and passed into child constructs. The child condition or statement does not independently determine where the surrounding program should branch.

6c. Cross-verification

Student B: the marker nonterminal does not change the language because M derives ε and consumes no input. Every terminal string generated by the original declaration grammar is still generated by the transformed grammar; M only introduces a semantic-action point.

Student A: code concatenation order matters because instructions must execute after the instructions that compute their operands. For x=a+b*c−d, the multiplication instruction must appear before the addition, and the addition before subtraction. Reordering fragments can use an uninitialised temporary or compute a different result.

Q7 – DAY-4 DESIGN CHALLENGE: SYNTAX-DIRECTED TRANSLATOR FROM FIRST PRINCIPLES (10 MARKS)

Stage 1 – Independent design proposals

Student A proposal: typed expression and declaration translator

Student A proposes a small language supporting typed declarations, arithmetic expressions and if-else statements. The grammar separates declarations from statements and uses synthesized expression types/values plus inherited declaration and control-flow context.

```
Program → DeclList StmtList
Decl    → Type IdList ;
Type    → int | float
IdList  → id | IdList , id
Stmt    → id = Expr ;
        | if ( Expr ) Stmt else Stmt
Expr    → Expr + Term | Expr - Term | Term
Term    → Term * Factor | Term / Factor | Factor
```

Factor → id | NUM | (Expr)

Attributes: Type.type is synthesized; IdList.in is inherited; Expr.type and Expr.addr are synthesized; Stmt.next and branch labels are inherited. The design uses left-to-right evaluation for declarations and bottom-up evaluation for expressions.

Student B proposal: AST and three-address-code translator

Student B proposes a translator that builds an AST first and then emits TAC. Each expression returns node, addr and code attributes. The declaration context supplies the type to each identifier. If-else statements receive true, false and next labels from the parent.

Expr → Expr1 + Term

Expr.node = mkNode("+", Expr1.node, Term.node)

Expr.addr = newTemp()

Expr.code = Expr1.code || Term.code ||

[Expr.addr = Expr1.addr + Term.addr]

Stmt → id = Expr ;

Stmt.code = Expr.code || [id = Expr.addr]

Stage 2 – Design-comparison record

Point	Student A proposal	Student B proposal	Joint decision
Main focus	Type checking and attribute flow.	AST construction and TAC generation.	Combine semantic checks with AST/TAC.
Expression values	Synthesized type/value attributes.	Node pointer, address and code.	Use synthesized type, node and address attributes.
Declarations	Inherited type passed into identifier list.	Same inherited declaration context.	Use L.in and addType routine.
Control flow	Inherited branch labels.	Labels emitted with TAC.	Use inherited true/false/next labels.
Evaluation	Left-to-right for L-attributed rules; post-order for expressions.	Build AST then emit code.	Use L-attributed front-end and post-order expression processing.

Agreements: both proposals use a grammar, inherited declaration context and synthesized expression results. Differences: A emphasises semantic validation, while B emphasises intermediate-code generation. The final design retains A's explicit type checking and B's AST/TAC construction. It also adds duplicate/undeclared-name checks and recoverable error reporting.

Stage 3 – Joint final design

Final grammar and attribute table

D → T L ;

T → int | float

L → id L'

L' → , id L' | ε

S → id = E ;

E → E + Tm | E - Tm | Tm

Tm → Tm * F | Tm / F | F

F → id | NUM | (E)

Attribute	Kind	Purpose
T.type	Synthesized	Type selected by int or float.
L.in / L'.in	Inherited	Type passed to every declared identifier.
E.node / Tm.node / F.node	Synthesized	Pointer to the AST subtree.
E.addr / Tm.addr / F.addr	Synthesized	Location holding the computed value.
E.code / Tm.code / F.code	Synthesized	TAC instruction sequence.
S.next / B.true / B.false	Inherited	Control-flow labels supplied by enclosing statement.

Joint SDT sketch

T → int { T.type = INTEGER; }

T → float { T.type = REAL; }

D → T { setCurrentType(T.type); } L ;

L → id { addType(id, currentType); } L'

L' → , id { addType(id, currentType); } L'1

L' → ε

F → id { F.node = mkLeaf(id); F.addr = id; }

F → NUM { F.node = mkLeaf(NUM); F.addr = NUM; }

E → E1 + Tm

{ E.node = mkNode("+", E1.node, Tm.node);
E.addr = newTemp();
E.code = E1.code || Tm.code ||
[E.addr = E1.addr + Tm.addr]; }

The final design is L-attributed for declaration flow because the type is known before the identifier list is processed. Expression attributes are synthesized. A parser can evaluate the declaration context left-to-right and expression attributes bottom-up.

Symbol table and error handling

Each symbol-table record contains identifier name, data type, scope, declaration line and optional storage location. addType checks for duplicates in the current scope. Identifier use performs lookup; a failed lookup reports an undeclared variable. Type checking rejects incompatible assignments and invalid operations such as division by a constant zero.

Implementation pseudocode

```
compile(source):
    tokens = lexical_analysis(source)
    tree = parse(tokens)
    if syntax errors exist: report and stop semantic translation
    evaluate declaration attributes left-to-right
    build AST nodes for expressions
    check identifier declarations and expression types
    if semantic errors exist: report them
    else:
        generate TAC from AST
        print symbol table, AST and TAC
```

Stage 4 – Validation: formal and experimental

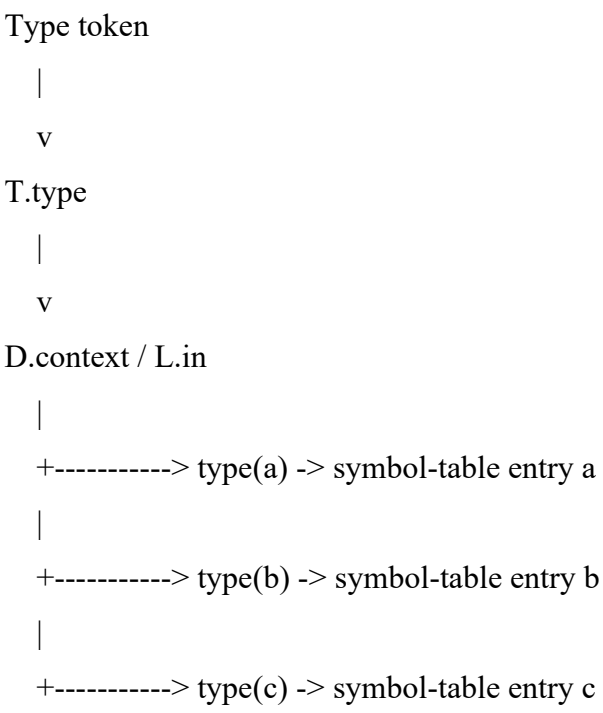
Student A – formal validation

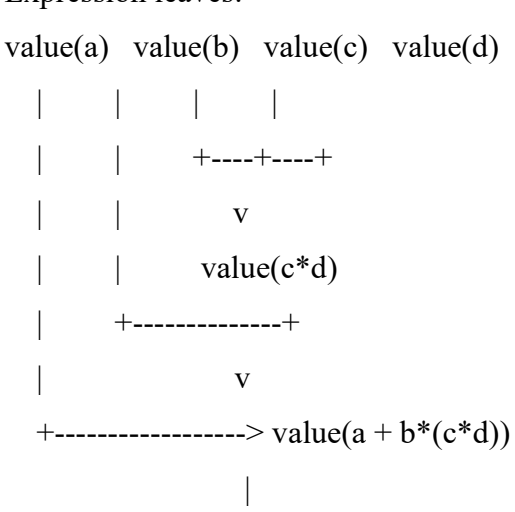
No-cycle argument for the expression SDD: every operator node depends only on child expression attributes. The parse tree is finite and edges point from lower child nodes toward their parent attributes. A

directed cycle would require a node to be an ancestor of itself, which is impossible in a tree. For declarations, T.type is computed first, then passed to L.in and down the identifier list. The dependency direction is strictly from the type node to the list and its entries, so no dependency points back to T.type. Thus the described attribute dependencies are acyclic.

The declaration design is L-attributed because each inherited attribute of a symbol uses the parent's inherited context or attributes of symbols to its left. Each identifier is processed once; symbol-table lookup/insert is expected O(1) with hashing, so processing n declarations is expected O(n). With a balanced search tree, the cost is O(n log n).

Full-program dependency graph





v

assignment TAC

Student B – experimental validation plan

Use the following correctness corpus. Expected outputs are calculated manually and should be compared with the program's actual output. No execution time is claimed here because timings must be measured on the machine used for the assignment.

Test	Input	Expected result
Valid declaration	float a,b,c;	a,b,c inserted as REAL.
Integer declaration	int count;	count inserted as INTEGER.
Duplicate	int a; float a;	Duplicate declaration reported in same scope.
Undeclared use	x = a + 1; with a absent	Undeclared identifier a reported.
Precedence	x = 2 + 3 * 4;	TAC computes 3*4 before addition; value 14.
Parentheses	x = (2+3)*4;	Addition before multiplication; value 20.
Associativity	x = 10-3-2;	(10-3)-2 = 5.
Division by zero	x = 5/(3-3);	Semantic/runtime safety error reported.
Syntax error	x = 2 + * 4;	Parser reports syntax error.
Type mismatch	int x; x = 2.5;	Type conversion/error policy applied.

Input size	Declarations / expressions	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Median (ms)
Small	10	____	____	____	____
Medium	100	____	____	____	____
Large	1000	____	____	____	____

Interpretation: correctness is established when every valid test produces the expected symbol table, AST and/or TAC, and every invalid test produces the expected diagnostic. Performance should be reported

from repeated measurements. The expected trend for a single-pass L-attributed front-end with hash-based symbol lookup is approximately linear in the number of tokens, excluding expensive external operations.

Stage 5 – Cross-defence

Student A explains the experimental evidence: valid declaration and expression cases must match their hand-derived results; invalid cases must produce diagnostics; timings are compared using the median of repeated trials. A timing claim is accepted only when the input size, hardware, compiler settings and repeated runs are recorded.

Student B explains the attributes: T.type is synthesized because it comes from the type keyword; L.in is inherited because the declaration context passes the type to identifiers; expression values and AST pointers are synthesized because they are constructed from child results; branch labels are inherited because the enclosing control-flow construct determines the destinations.

Stage 6 – Joint evaluation and designer conclusion

SDT is suitable when translation actions can be performed at predictable points during parsing, such as emitting postfix operators after operands or recording a declaration type before processing identifiers. It saves a separate traversal when the output can be produced safely during parsing.

Costs include embedded actions that can make grammar specifications harder to read, dependence on action placement, reduced flexibility for later optimisation, and more complex error recovery if output is emitted too early. SDT is not ideal when the whole program must be analysed before translation, when global optimisation is required, or when semantic meaning depends on information discovered much later.

- Define the grammar and precedence before writing semantic actions.
- Classify every attribute as synthesized or inherited.
- Draw dependency arrows and verify that no cycle exists.
- Ensure inherited attributes are available before they are read.
- Use explicit symbol-table checks for duplicate and undeclared identifiers.
- Test valid, invalid, boundary and nested expressions.
- Compare program output with hand calculations and preserve reproducible test data.
- Measure performance instead of inventing timing results.

Designer conclusion: the joint translator combines the strengths of S-attributed expression evaluation and L-attributed declaration processing. It builds a compact AST, checks declarations and types, and generates TAC. The design is understandable, extensible and suitable as a teaching-scale compiler front-end.

CONCLUSION

This assignment demonstrates syntax-directed translation using synthesized and inherited attributes. Arithmetic expressions were evaluated with an S-attributed definition, while declarations used inherited type information in an L-attributed definition. Dependency graphs established legal evaluation orders and exposed cyclic definitions. Syntax-directed translation schemes generated postfix output at suitable action positions. AST construction reduced grammar-specific parse-tree structure to meaningful operators and operands. Finally, a joint translator design combined declaration checking, AST construction and three-address-code generation. The work shows that correct attribute classification, dependency analysis, action placement and systematic testing are essential for a reliable compiler.

REFERENCES

13. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D., Compilers: Principles, Techniques, and Tools, 2nd Edition, Pearson, 2006.
14. Cooper, K. D., and Torczon, L., Engineering a Compiler, 2nd Edition, Morgan Kaufmann, 2011.
15. Parr, T., The Definitive ANTLR 4 Reference, 2nd Edition, Pragmatic Bookshelf, 2013.
16. GNU Bison Manual, sections on semantic values, actions and parser rules.
17. Course notes: Compiler Design – Unit III, Syntax-Directed Translation, SIMATS Engineering.